**Detailed Analysis Report on Sorting Algorithms**

## Introduction

Sorting algorithms are essential when it comes to organizing data usually from the smallest to greatest value. In this assignment we implement the usage of Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, and Merge Sort. This assignment's primary focus is on determining the execution time in nanoseconds and the swaps that are performed by each algorithm in order to arrange the numbers in the desired order. The larger the data set we have the more important these two factors are when working with the sorting algorithms.

## Brief Description of each Sorting Algorithm

1. Bubble Sort:
    - Bubble Sort compares adjacent elements and swaps them if they are not in the correct order, this process is repeated until there are no more swaps necessary. When there are no more swaps required that shows the list has been sorted.
2. Insertion Sort:
    - Insertion Sort is split into two halves, the sorted (left) and the unsorted (right). This algorithm builds the sorted array by adding an element from the unsorted section into the correct location of the sorted section.
3. Selection Sort:
    - Selection Sort finds the minimum value in the array and swaps it with the lowest unsorted element.
4. Shell Sort
    - Shell Sort reduces the gap between elements that are being compared, initially it compares elements that are further apart from each other. Eventually the gap decreases and this process acts like an Insertion Sort.
5. Merge Sort
    - Merge Sort recursively divides the array into two halves until every element is by itself. Afterwards it recursively sorts each half and merges the sorted halves back together.

## Implementation Approach

The approach taken to implement all of these recursive sorting algorithms was to first create an array of a random size between 50 and 5000. Once this method ran and there was an established length of the array, there was another method called which created the array and filled it with the appropriate amount of elements that entirely filled the array. Afterwards each sorting algorithm was implemented in the program and also called in the main method. In order to prevent a sorted

array to be re-sorted and the unsorted array was cloned so that each sorting algorithm can have an opportunity to sort the array at their own efficiency. In the main method each one of the sorting algorithms got called and an integer “swaps” was implemented to each algorithm to determine efficiency of storage later down the road when analyzing the efficiency. There was also another component implemented that measured the time, in nanoseconds, to determine how efficient the algorithm is with the given data set. At the very end there was also a little segment added which compared each array with the use of Array.equals(). For this line segment, if each array is equal to each other then a final version of the array is printed which is the completely sorted array. Lastly, to create the graph needed for an accurate representation of each Sorting Algorithm, the generate function was temporarily commented out and the arrLength was manually set to 50 and incremented to 5000 manually. After each increment the data was copied to a spreadsheet where a graph was made.

**Addition Comparison: Impact of Number of Exchanges on Algorithm Performance**

Other than time complexity, the number of swaps also plays a very important role in performance, especially when taking into account factors like memory usage and efficiency. While time complexity shows how quick an algorithm works with large data sets, the number of swaps can help show the tradeoffs of certain algorithms where the time can be effective, however it can take up an immense amount of space.

Bubble Sort

- Bubble sort tends to make a significant number of swaps, especially an array that isn’t anywhere near being sorted. This high number of swaps results in a high execution time and also requires more memory in order to store intermediate values during the swaps. This makes Bubble Sort inefficient in terms of both time and space with the larger the data set gets. Overall, this sorting algorithm is less effective than the Insertion Sort which is more efficient with memory usage compared to Bubble Sort.

Insertion Sort:

- The number of exchanges that are in an InsertionSort depends heavily on the initial order of data with the program being in reverse order being the worst case. This sorting algorithm gets worse as the number of swaps increases but it is more efficient than Bubble Sort due to its shifting elements and not making an abundant amount of swaps. When comparing the two the insertion sort is more efficient when it comes to memory usage due to having a lot less swaps than Bubble Sort. However, they both aren’t ideal for larger sets of data.

Selection Sort

- This algorithm performs less swaps than the two prior Sorts mentioned. This results in Selection Sort being the better option when swaps and storage becomes a priority. Despite having less swaps, the time complexity for this algorithm is still not efficient for data sets that are larger.

Shell Sort

- The number of swaps in this algorithm is dependent on the gap sequence. The better the gap sequence is the closer the algorithm will be to having a time complexity of O(n logn). The trade off for this algorithm is heavily dependent on the gap with a good gap providing better performance in time complexity and in number of swaps. With a bad gap sequence both the performance in time complexity and the number of swaps.

Merge Sort

- Due to the approach of divide and conquer, this algorithm does well with swaps. With this algorithm being on the smaller side of swaps, this results in a higher efficiency even when the data set is large. When considering time complexity and number of swaps this algorithm is very efficient. The only trade off with this algorithm is that it has more memory used due to the merging.

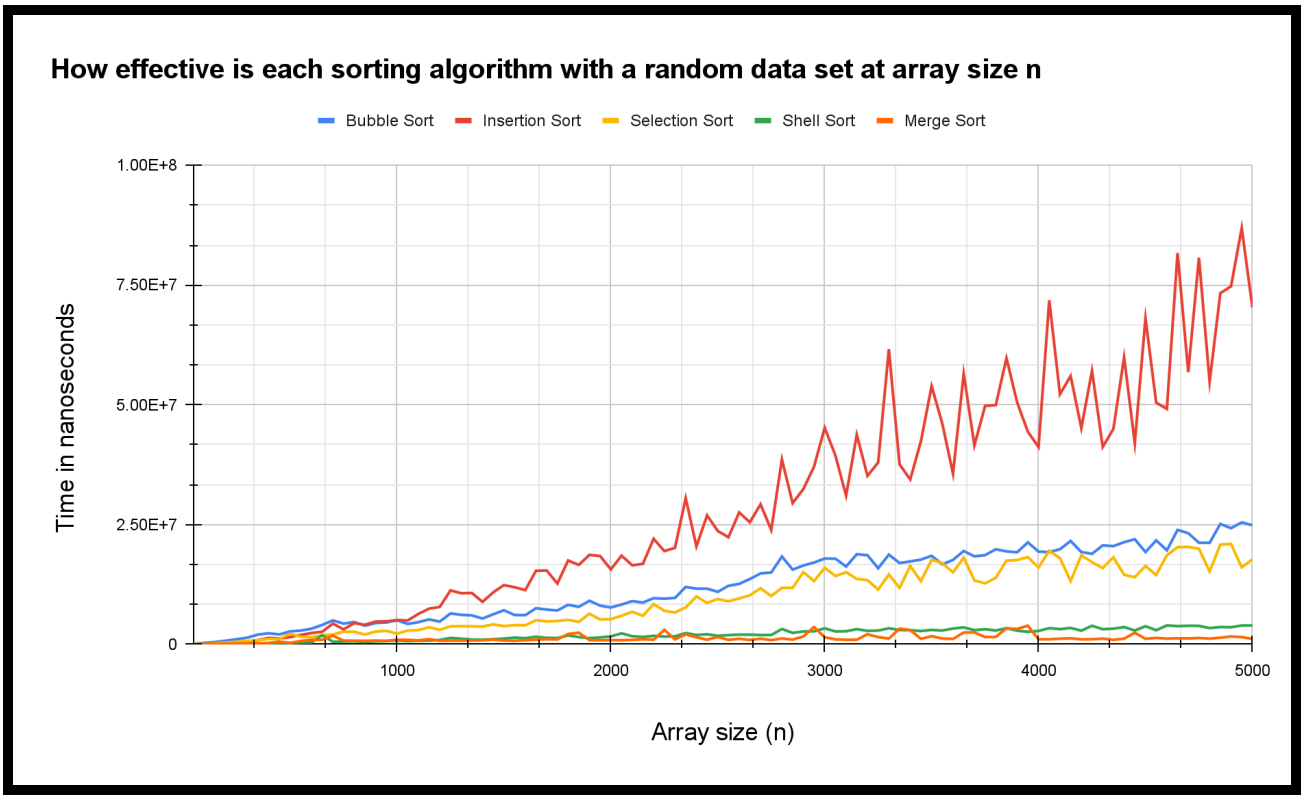
**Performance Analysis**

- Bubble Sort:
    - Execution Time:
        - Due to the time complexity being O(n^2), the larger the array gets the longer it will take for the data set to be sorted.
    - Swaps:
        - This sorting algorithm requires a large number of swaps, especially if the array is far from being sorted due to the algorithm only swapping the elements adjacent and with elements that reach 5000 that can severely slow down the program and also require more swaps.
- Insertion Sort:
    - Execution Time:
        - This sorting algorithm also takes longer to sort arrays of larger size due to it having a time complexity of O(n^2).
    - Swaps:
        - The number of swaps that is required for this algorithm depends on how many elements there are that are out of order.
- Selection Sort:
    - Execution Time:

        - This sorting algorithm also has a time complexity of O(n^2), due to this in terms of execution time it performs similarly to both bubble and insertion sort.
    - Swaps:
        - Despite having the same time complexity as the two previously mentioned algorithms, there are fewer swaps required due to the fact that this algorithm only swaps once per iteration of the for loop.
- Shell Sort:
    - Execution Time:
        - With it having a time complexity of O(n logn) this sorting algorithm is one of the more efficient ones however this is heavily dependent on the gap sequence. If the gap sequences are not good this could result in the algorithm having a time complexity of O(n logn).
    - Swaps:
        - The efficiency of the swaps depends on the chosen gap sequence, however in general it is more efficient than the O(n^2) algorithms that don't have the opportunity to be O(n logn).
- Merge Sort:
    - Execution Time:
        - For datasets of this size and larger this algorithm had the best execution time overall due to it always having a time complexity of O(n log n).
    - Swaps:
        - This sorting algorithm only performs swaps when it is combining the elements that have been already split into singular elements.

**Graph Analysis**

Upon completion of the graph it is clear which sorting algorithm is the worst in terms of time and which ones are better of the bunch. By a landslide the Insertion Sort is the least effective out of all of them and is also very inconsistent based on the data set it is presented. Although the data that is being sorted is random with every attempt, that shouldnt take away from the fact that this Sorting algorithm is not very effective. Both Bubble Sort and Selection Sort were close in time complexity with Bubble Sort being slightly worse overall. The Bubble Sort and Selection Sort stuck pretty close to one another with the Selection Sort having a little more dramatic dips when it came to more optimal randomly generated data sets. The best two Sorting algorithms were the Merge Sort and Shell Sort with the Merge Sort being slightly better for the data sets and possible. There were occasional instances where the Merge Sort's time complexity surpassed the Shell Sort however this occurred when the formation of the data sets was very unfortunate towards the Merge Sort and overall favored the Merge Sort as the best Sorting algorithm.




**Conclusion**

After analyzing the five sorting algorithms I've noticed that Merge Sort is the most efficient algorithm overall, it performs well with larger data sets such as the ones seen in the program. This is due to its time complexity being O(n log n). Although it does require a little more memory for temporary arrays that are used in the merge process, it minimizes the time required to sort large arrays. Shell Sort is also a good option as it can be O(n log n) but also O(n^2). Data sets in the range of 50 and 5,000 seem to be fairly small and the difference between Merge Sort and Shell Sort isn't great but Merge Sort is Still better. For larger arrays such as the ones presented, Bubble Sort, Insertion Sort, and Selection Sort all have a time complexity of O(n^2) however the Insertion Sort is much more inconsistent and breaks off from the other two Sorting methods in time taken to complete. These three Sorts, especially Insertion Sort, should be ruled out for data sets of such size as the difference is notable and as we continue to get larger sets the worse they become. Selection Sort is one of the most efficient algorithms for these data sets when limiting the amount of swaps is a priority. Otherwise, Selection Sort isn't very efficient for larger data sets or in general because it will always have a time complexity of O(n^2). Overall, Merge Sort is the best algorithm due to it having a good performance in both execution time and swaps. Shell Sort is a close competitor with Merge Sort however it is slightly worse. The further out data sets get from 50, the worse Bubble Sort, Insertion Sort, and Selection Sort become. This should rule out these three options for data sets of this size.